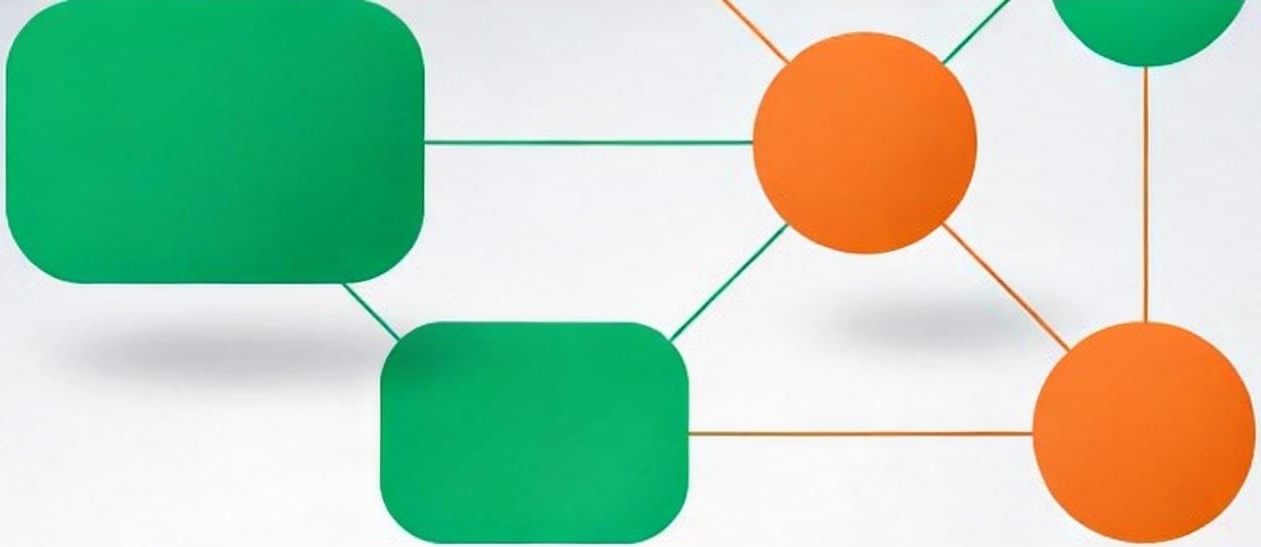
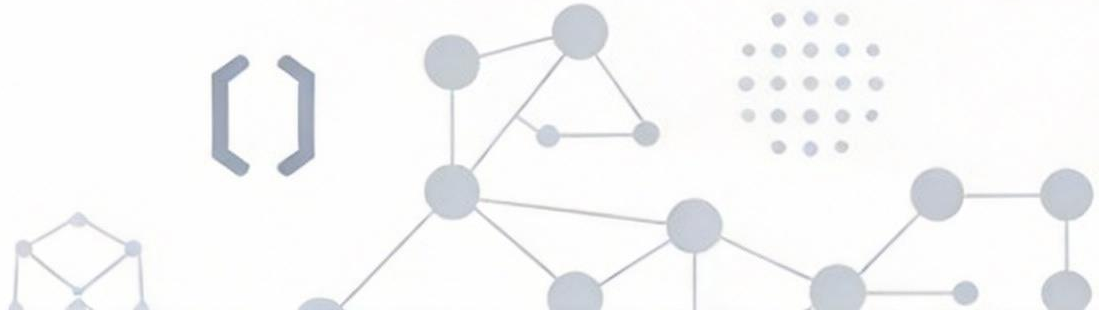


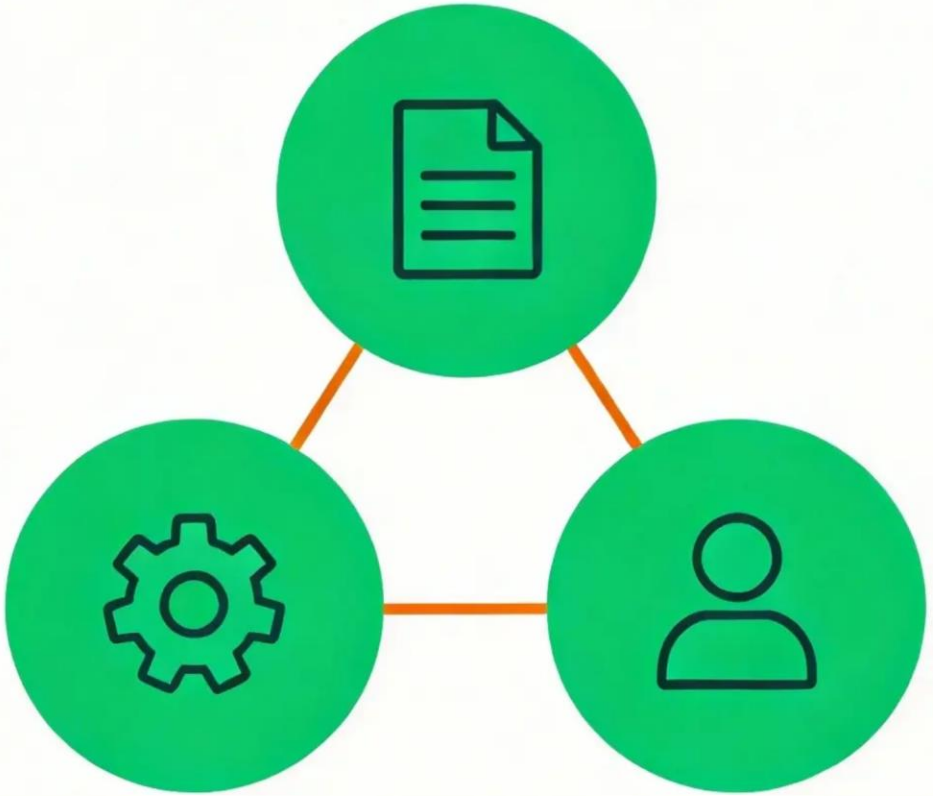
Programación Orientada a Objetos (POO) y Principios SOLID



¿Qué es la Programación Orientada a Objetos?

La POO es un paradigma de programación que organiza el software como una colección de objetos interconectados.
Cada objeto combina datos (atributos) y funcionalidades (métodos) en unidades reutilizables llamadas clases.

Concepto Clave:
Los objetos encapsulan datos y comportamientos, permitiendo crear software modular y mantenible



Paradigma Moderno

¿Por qué es Importante la POO?

Reutilización de Código

Reduce tiempo y esfuerzo siguiendo el principio DRY

Mantenibilidad

Cambios localizados con impacto mínimo en el sistema

Modularidad

División en componentes manejables y organizados

Ampliamente Usado

Presente en la mayoria de lenguajes de programación modernos

Flexibilidad y Escalabilidad

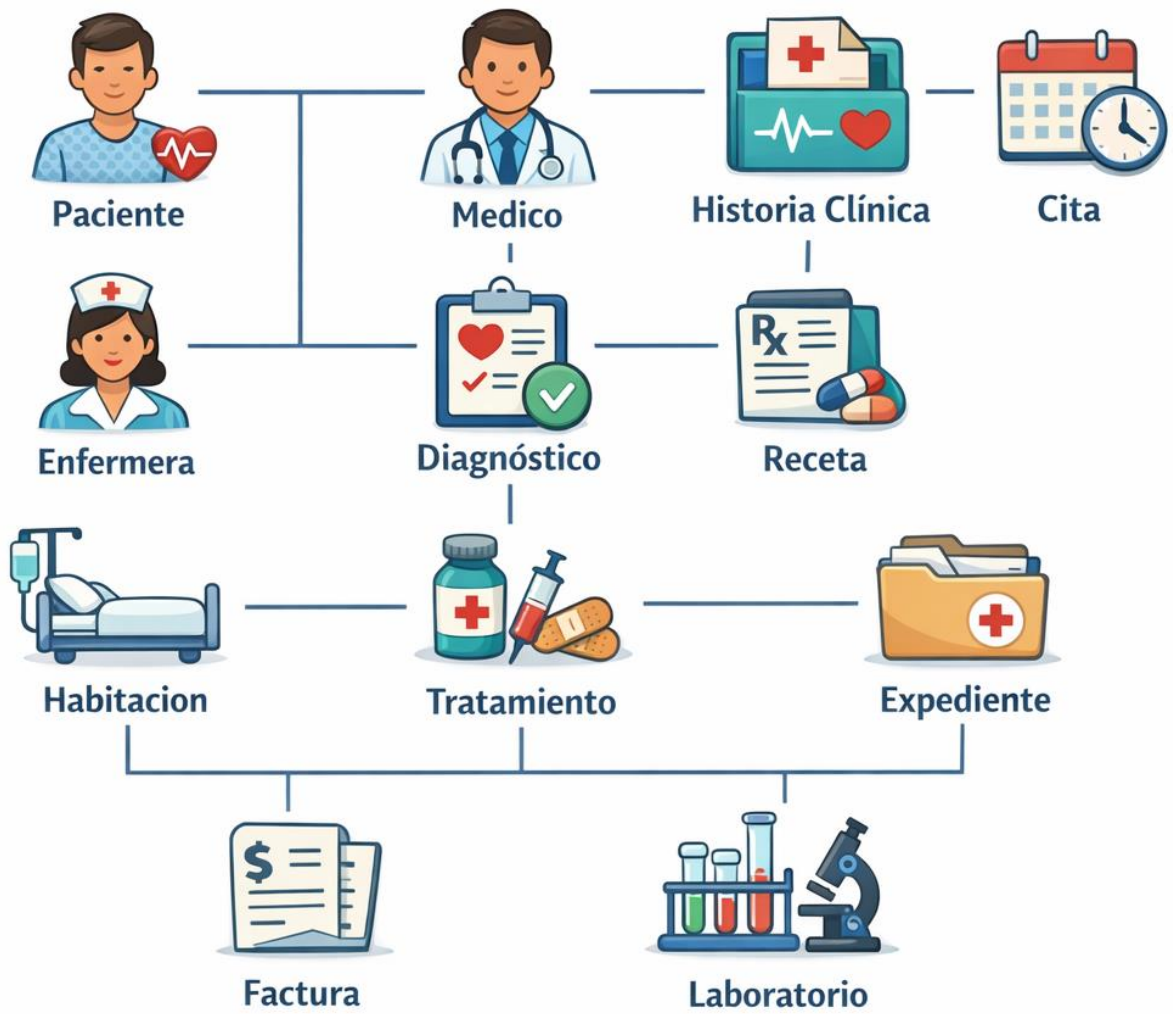
Adaptable a cambios futuros sin reestructuración

Protección de Información

Encapsulamiento que aumenta seguridad y fiabilidad

La POO facilita el desarrollo de software robusto, mantenible y escalable

Objetos en un Sistema de Hospital



Conceptos Fundamentales de la POO

Clase

Plantilla que define cómo serán los objetos de un tipo determinado. Especifica atributos y métodos.

Objeto

Instancia concreta de una clase con atributos definidos independientemente.

Atributo

Características que describen el estado de un objeto. Representan los datos.

Método

Acciones que un objeto puede realizar. Operan sobre los atributos.

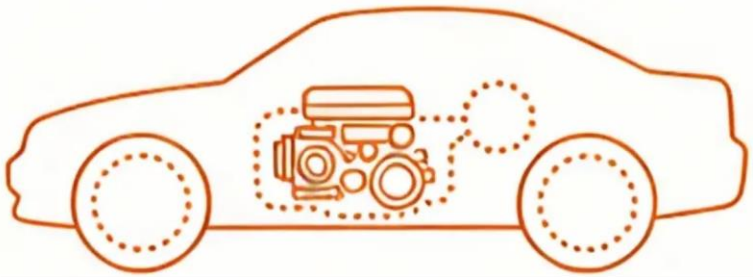
Los 4 Pilares de la POO - Encapsulamiento y Abstracción

ENCAPSULAMIENTO

Agrupa datos y métodos dentro de una única unidad (clase). Oculta los detalles internos y expone solo una interfaz pública para proteger el estado interno del objeto.

ABSTRACCIÓN

Muestra solo las características esenciales, ocultando los detalles complejos de implementación. Permite trabajar con conceptos a un nivel superior.

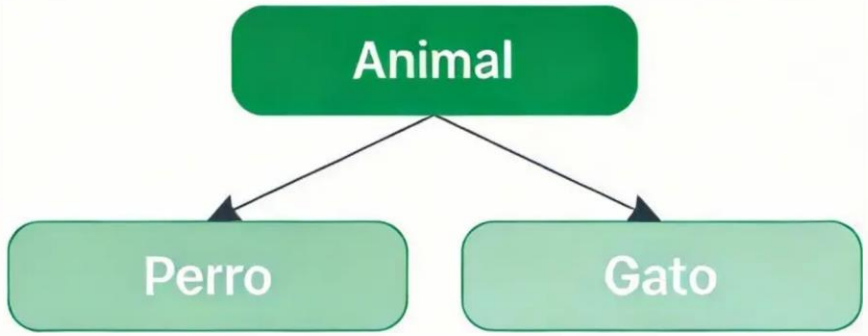


Ejemplo: Al conducir un coche, usas el volante y pedales (interfaz pública) sin conocer los mecanismos internos del motor (detalles encapsulados).

Los 4 Pilares de la POO - Herencia y Polimorfismo

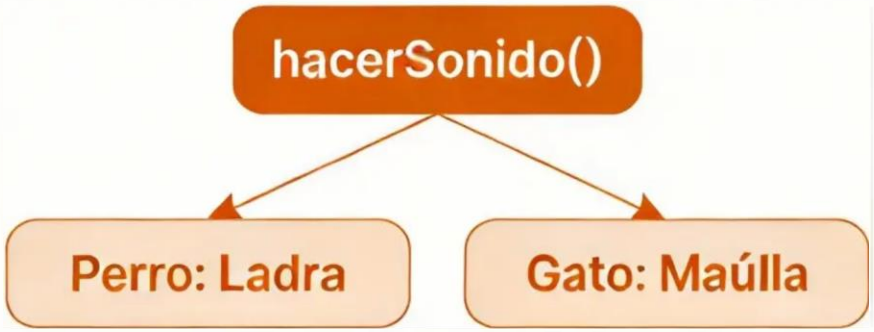
HERENCIA

Permite que una clase hija adquiera propiedades y métodos de una clase padre, estableciendo relaciones jerárquicas tipo 'es un/a' y promoviendo la reutilización de código.

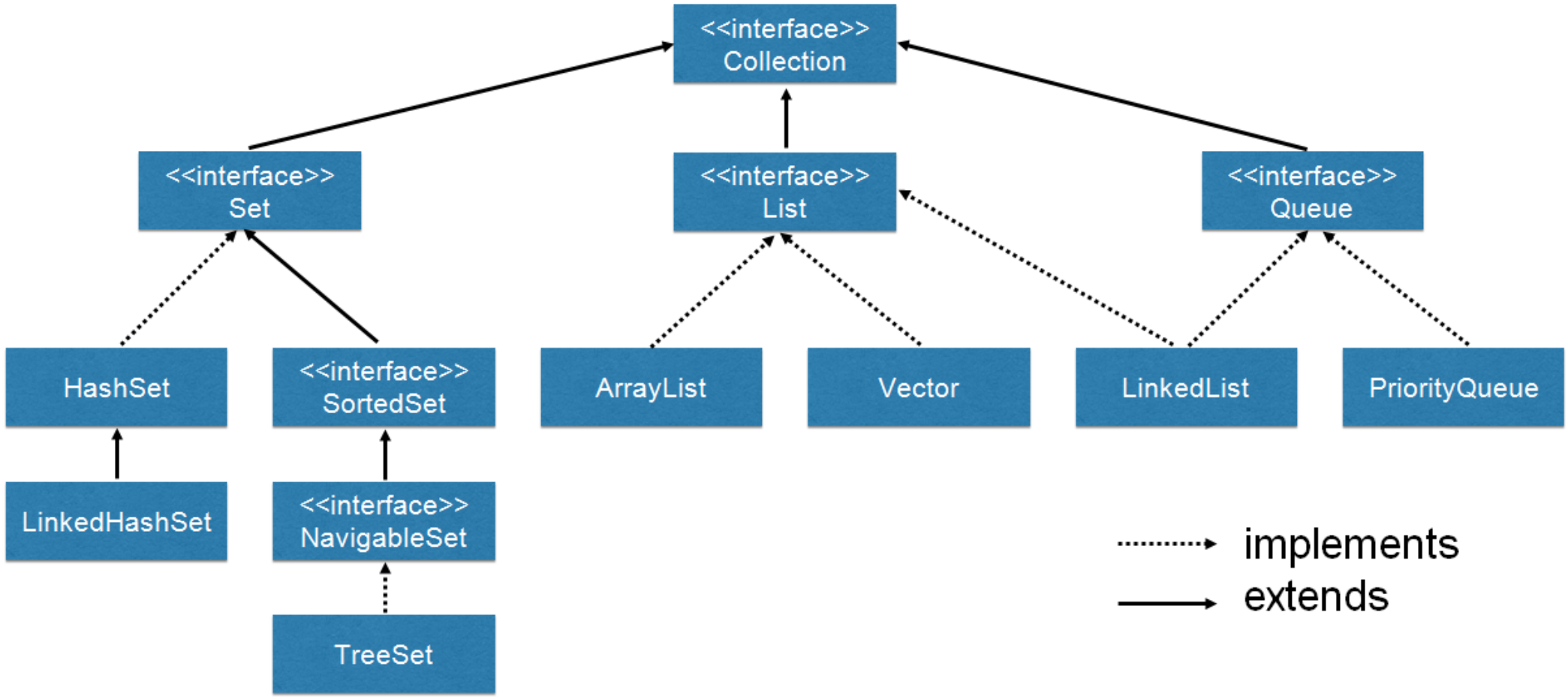


POLIMORFISMO

Capacidad de objetos de diferentes clases de responder al mismo método de maneras distintas. Una misma interfaz puede tener múltiples implementaciones específicas.



Collection Interface



Introducción a los Principios SOLID

Los principios SOLID son cinco directrices de diseño que extienden los conceptos de POO para crear soft-ware más comprensible, flexible y mantenible.

Objetivo: Mejorar la calidad del código, la extensibilidad y la flexibilidad

- **S** - Single Responsibility Principle
- **O** - Open/Closed Principle
- **L** - Liskov Substitution Principle
- **I** - Interface Segregation Principle
- **D** - Dependency Inversion Principle

Principios S, O y L de SOLID

S

Responsabilidad Única

Una clase debe tener una sola razón para cambiar. Ejemplo: Separar lógica de negocio, impresión y persistencia en clases diferentes.

O

Abierto/Cerrado

Abierto para extensión, cerrado para modificación. Ejemplo: Usar interfaces para añadir nuevos tipos sin modificar código existente.

L

Sustitución de Liskov

Las subclases deben ser sustituibles por sus clases base sin alterar el funcionamiento.

Principios I y D de SOLID

I - Segregación de Interfaces

Es mejor tener muchas interfaces específicas que una interfaz general. Ninguna clase debe implementar métodos que no utiliza.

D - Inversión de Dependencia

Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones.

Ejemplo: PedidoService depende de interfaz IPedidoRepository, no de implementación concreta

Beneficio: Mayor flexibilidad y desacoplamiento del código

Conclusión - POO y SOLID en el Desarrollo Moderno

La POO y los principios SOLID constituyen pilares fundamentales del desarrollo de software moderno. Su aplicación conjunta es crucial para construir aplicaciones de alta calidad que puedan adaptarse y evolucionar en el dinámico panorama tecnológico actual.

Código Limpio y Mantenible

Sistemas Escalables y Flexibles

Desarrollo Eficiente

¡Gracias!

What’s Next

Patrones de Diseño
Patrones de Arquitectura
Monolito vs Microservicios